

An expected coverage model with a cutoff priority queue

Soovin Yoon¹ · Laura A. Albert¹ 

Received: 6 March 2017 / Accepted: 3 July 2017 / Published online: 28 July 2017
© Springer Science+Business Media, LLC 2017

Abstract Emergency medical services provide immediate care to patients with various types of needs. When the system is congested, the response to urgent emergency calls can be delayed. To address this issue, we propose a spatial Hypercube approximation model with a cutoff priority queue that estimates performance measures for a system where some servers are reserved exclusively for high priority calls when the system is congested. In the cutoff priority queue, low priority calls are not immediately served—they are either lost or entered into a queue—whenever the number of busy ambulances is equal to or greater than the cutoff. The spatial Hypercube approximation model can be used to evaluate the design of public safety systems that employ a cutoff priority queue. A mixed integer linear programming model uses the Hypercube model to identify deployment and dispatch decisions in a cutoff priority queue paradigm. Our computational study suggests that the improvement in the expected coverage is significant when the cutoff is imposed, and it elucidates the tradeoff between the coverage improvement and the cost to low-priority calls that are “lost” when using a cutoff. Finally, we present a method for selecting the cutoff value for a system based on the relative importance of low-priority calls to high-priority calls.

Keywords Coverage model · Cutoff priority queue · Emergency medical services · ambulance location · Hypercube approximation · Simulation

1 Introduction

Emergency medical service (EMS) systems provide pre-hospital care to those who are in need of urgent medical treatment and transports these patients to hospitals. The promptness of response is critical for many patient conditions such as cardiac arrest, since the condition of the patient can deteriorate when the service is delayed. As a result, the design of public safety systems is crucial for effectively using scarce emergency medical resources for responding to patients in a timely manner. In particular, patients with time-critical conditions who require immediate care are more vulnerable to the delay of service resulting from congestion in the system. This motivates the development of response plans that depend on the level of available resources in the system as well as the specific needs of the patients.

We consider a system with multiple ambulances and multiple patient priority classes. We apply the commonly used *coverage* performance measure, under which a call is considered to be *covered* when an ambulance reaches the patient in a given period of time, e.g., nine minutes from dispatch. Although the ultimate goal of emergency medical services is to provide prompt service to all patients, available emergency medical personnel and ambulances are limited, and therefore, triage may be necessary to better match resources with patients. Calls with higher priorities are more time-sensitive, while other accident types with lower priorities can wait without significant deterioration in patient conditions. For example, the chance of survival for cardiac arrest patients decreases by up to 10% for every

✉ Laura A. Albert
laura@engr.wisc.edu

Soovin Yoon
yoon57@wisc.edu

¹ Industrial & Systems Engineering Department,
University of Wisconsin-Madison, Madison,
53706, USA

additional minute of delay in treatment [24]. Therefore it is important to efficiently ration limited resources, not only between different geographic locations but also among call priorities [29].

In this paper, we study how to give priority to more emergent calls for service when the system is congested by reserving capacity for high priority calls, which is called the *cutoff priority scheme*. Existing ambulance location models generally assume that all calls are immediately served as long as there is an available vehicle [16]. We lift this assumption by adopting a cutoff priority scheme. We limit our analysis to a two-priority case in this paper and hence we impose a single cutoff number for low-priority calls. In our model, a cutoff level is given for admission of low priority customers, so that low priority calls are either delayed or diverted to neighboring EMS departments (a process called “mutual aid”) whenever the number of busy ambulances is equal to or greater than the cutoff. Therefore we always reserve a subset of servers exclusively to serve high priority patients in a timely manner. We also study system design by locating and dispatching ambulances to prioritized patients when there is a low priority cutoff. Mutual aid policies between adjacent EMS departments is commonplace in the United States, and queueing nonurgent emergency calls when the number of available servers fall below the threshold is a strategy in use in Europe [3], however, the cutoff priority scheme has not been studied in the literature in the emergency response context. Under the cutoff priority scheme, the optimal system design differs from the design without the cutoff, which leads to improved coverage of emergency calls.

There is a rich operations research literature devoted to the design and evaluation of public safety systems such as fire and EMS. Early models in this area study how to locate vehicles such as ambulances to maximize coverage [11], the most common performance measure for EMS systems. A number of papers consider the impact of congestion and backup service when locating vehicles. Daskin [12] introduces one of the first probabilistic location models that takes ambulance unavailability into account to maximize expected coverage by assuming that each ambulance has the same busy probability of being unavailable for service. Batta et al. [4] extend this Daskin’s expected coverage model to account for the underlying spatial queueing dynamics of the ambulances using a Hypercube queueing model. Other models that locate ambulances to maximize expected coverage allow for multiple types of ambulances [28] and multiple ambulances per station [1]. Restrepo et al. [33] present two models based on the Erlang loss formula that can be used to screen potential ambulance allocations. There is also a stream of literature for ambulance redeployment [14, 23, 27, 32]. However, all of these models assume that an ambulance is sent to a call if one is available and

therefore do not ration service as we consider in this paper. For a more extensive review on ambulance location models, we refer the readers to the review article by Brotcorne et al. [7], and two recent reviews by Aringhieri et al. [2] and Reuter-Oppermann et al. [34].

There is a stream of papers that develop spatial queueing models for evaluating the design of public safety systems. Larson develops an exact Hypercube queueing model to study the behavior of servers such as ambulances or fire engines [25] and provides an approximate Hypercube model that approximates the procedure by computing correction factors [26]. Jarvis [21] generalizes the approximate Hypercube model of [26] to account for location-specific service times and customer (patient) priorities. There are a variety of other models that lift the assumption in earlier Hypercube models to allow more practical settings [6, 9, 17, 31, 36]. These Hypercube models can evaluate various performance measures once a system has been designed, but they cannot construct the optimal system configuration. Instead Hypercube models are used to approximate the inputs for many location models introduced earlier, e.g., [4, 28] and [1]. One exception is Geroliminis et al. [15] who develop a spatial queueing model that considers spatial characteristics and determines location and dispatch decisions internally by optimization. Another is Cho et al. [10], who devise an integrated method with iterative problem relaxations and restrictions to endogenously estimate the ambulance busy probabilities in a nonlinear optimization model.

The models considered thus far do not consider cutoff priority queues. The cutoff priority model was first formulated by Benn [5] and detailed by Jaiswal [20], who analyzes many cutoff priority model variations and their solutions. Taylor and Templeton [37] improve Jaiswal’s derivation for the cutoff priority model and solves balance equations to derive steady-state probabilities and waiting time distributions. They investigate two versions of the model: the SCQQ model with an infinite capacity queue for high priority calls and the SCQL model with a zero capacity queue for high priority calls. Both SCQQ and SCQL model maintain an infinite capacity queue for low priority calls. They apply the model to urban ambulance systems with two call priorities. Shaack and Larson [35] take a decomposition approach that solves the system for any number of priority classes to obtain the customer waiting time distribution. These cutoff priority queueing models provide steady-state probabilities for the number of busy servers, which are one of the building blocks for our spatial queueing model. The relationship between these cutoff priority models to our work is detailed in Section 2 and Appendix A.

In this paper, we introduce a spatial queueing model and a Mixed Integer Linear Program (MILP) for evaluating and designing public safety systems with a cutoff priority scheme. A structural overview of the models is illustrated in

Fig. 1. The cutoff reserves capacity for high priority calls so that service is not delayed for critical emergency medical patients when the system is congested. In our models, the number of ambulances is fixed, and our performance measure is the expected coverage. To the best of our knowledge, this paper is the first to explicitly exploit the issue of differentiating the queueing policy for different priority demands in emergency systems and use the cutoff to inform system design.

This paper makes the following contributions.

1. We derive the approximate Hypercube queueing model with a cutoff priority scheme that is computationally easy to implement and solve. We demonstrate the accuracy of the model using discrete event simulation.
2. We use the the Hypercube model to prescribe how to locate and send ambulances to calls for service by using the approximate Hypercube model to estimate input parameters for a MILP model we define. Solutions to the MILP model produce deployment and dispatching policies, which update inputs to the approximate Hypercube model. The two procedures alternatively exchange inputs and outputs until they reach convergence or infeasibility.
3. Computational examples illustrate how to implement the iterative procedure with real-world data. The results quantify the improvement in the expected coverage for high-priority patients using different levels of a cutoff. We demonstrate how to use the model to evaluate the tradeoff between expected coverage and the probability of losing or delaying calls when the cutoff is adjusted to help decision-makers set the value of the cutoff. The analysis illustrates how the number of reserved servers should decrease when the relative importance of low priority to high priority calls increases.

The remainder of this paper is organized as follows. Section 2 defines the notation and describes the approximate Hypercube model with cutoff priority queue. In Section 3 the MILP model is introduced for locating and dispatching

ambulances in an EMS system with a low priority cutoff. We describe the dataset and present computational results in Section 4 that validate the proposed models and report the model results. Then we conclude in Section 5.

2 Approximate hypercube model for cutoff priority queue

In this section, we propose an approximate Hypercube model with cutoff priority queue. The main goal of the model is to estimate the performance measures including the server busy probabilities, dispatch probabilities, and steady-state probabilities for the number of busy servers through an approximation that is precise and computationally efficient.

We implement two novel features. First, calls are distinguished not only by their locations but also by their priorities. We assume that a call taker performs triage on each call and preassigns one of two priorities to each call depending on the nature of the emergency, either high priority or low priority. Calls with a high priority are more urgent and require a faster response. Next, low priority calls are cut off—either lost or entered into a queue—when the number of busy servers exceeds a threshold in order to reserve servers for high priority call arrivals when the system is congested.

A system with s servers and high (low) priority calls from a set of geographical locations J is considered. Servers are located at their home stations, with one server at a station. Since only a subset of stations has a server located, those stations are denoted as open stations. While servers have identical capabilities, they are distinguished by their home station. The models can allow for multiple servers per location, when there is a preference ordering for dispatching the servers at the same station to calls for service. High (low) priority calls arrive independent of the busy status of the servers according to a Poisson process with rate $\lambda^H(\lambda^L)$ per hour. The Poisson arrival assumption is frequently consistent with real-world datasets [22, 36]. Service is nonpreemptive, and the average service time for calls from j served by server i is τ_{ij} , which includes the travel time from the station to the scene, the time spent at the scene, the time to transport the patient to the hospital and return to the station. We assume exponential service times but this can be relaxed if the system is a Loss system [21]. Exactly one server is assigned to a call (when one is assigned) according to a fixed preference list, which is an ordered list of the open stations following the cutoff scheme that is described later. The k th preferred station for the high (low) priority call from node j is given as $b_{jk}^H(b_{jk}^L) \in I^{open}$ in the preference lists $\{b_{jk}^H\}(\{b_{jk}^L\})$. For example, if $b_{12}^H = 3$, then the second preferred server for a call from node 1 is a server in station 3 if the call is high priority.

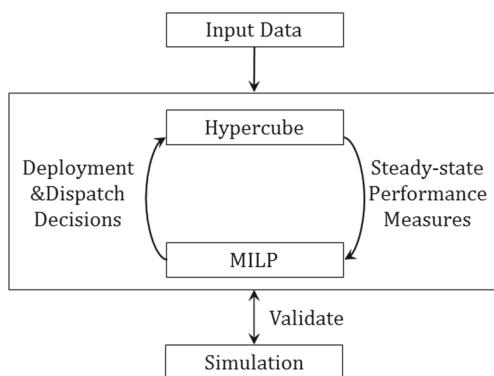


Fig. 1 Overview of the model structure

A cutoff level of s_C is given for admission of low priority customers, with $s_C \leq s$. This means that low priority calls are either placed in a queue or lost whenever there are s_C or more busy servers, instead of being served immediately by the most preferred available server. High priority calls are immediately served by the most preferred available server and either put in a queue or lost only if all servers are busy. This implies that the cutoff level for high priority customers is set to s by convention [35]. In other words, $s_R = s - s_C$ servers are always reserved exclusively for high priority calls to be prepared for the future arrival.

We consider two types of model that differ by the queue capacity. In the *Loss model*, high priority calls that arrive when all servers are busy or low priority calls that arrive when there are more than or equal to s_C busy servers are “lost,” and the queue has zero capacity ($M/M/s/s$). Lost calls are assumed to be served by external resources, such as neighboring EMS departments through a process called “mutual aid.” We also consider a *Queued model*, where the queue is assumed to have infinite capacity ($M/M/s/\infty$). Calls are put in a queue when they cannot be served immediately when all servers are busy for high-priority calls and when there are no more than s_R servers available for low-priority calls. Calls are served later on a first-come-first-served basis when servers become free. In this model, high priority calls in the queue are served as soon as a server becomes free, and queued low priority calls are served when the system has more than s_R available servers. Because the queue is infinitely large, backlogged calls are never lost as they are eventually served if the system is stable. If the cutoff is set to an extremely low value in a relatively congested system, the system may not be stable for the low priority calls, and we revisit this case later in a computational study.

Here we show the derivation for both systems. Throughout the paper we mostly assume the Loss system, since it is common to assume a zero-line queue in the EMS literature, and it is also consistent with real-world operations [8].

Table 1 summarizes the notation used in this paper. The Hypercube queueing model requires deployment and dispatch policies as inputs, which can be generated either externally (for example, send-the-closest) or by forming the preference lists from the optimal solutions of the MILP model in Section 3. Other parameters needed such as service times τ_{ij} and arrival rates λ_j^p can be collected from the data. We present derivation details for the results presented in this section in Appendix A.

The outputs of the approximate Hypercube model are the correction factors q_k^p , $k = 0, \dots, s-1$, $p \in \{H, L\}$, server busy probabilities r_i , $i \in I^{open}$, and steady-state probabilities P_i , $i = 0, \dots, s$. These outputs are obtained through an iterative procedure that we introduce later in this section.

The correction factors q_k^p adjust the errors from treating servers as independent. To be specific about what correction factors do, let B_k denote a event that j th selected server is busy and $F_k \equiv B_k^c$ denote a event that j th server selected is free. Then f_{jk}^p , the probability that a priority p call from node j is served by its k th preferred server, can be expressed as $P\{B_1 B_2 \cdots B_{k-1} F_k\}$. If the servers operate independently from one another, then the formulation for this term would be $\prod_{l=1}^{k-1} r_{b_{jl}}^p (1 - r_{b_{jk}}^p)$. However, the independence assumption is erroneous because the busy status of a server affects the probability of other servers being busy. Therefore, correction factors q_k^p , which we derive later, are needed to improve this approximation. Once we have the correction factors and server busy probabilities r_i , $i \in I^{open}$, the dispatch probability f_{jk}^p , which is the probability that a priority p call from node j is served by its k th preferred server, can be approximated as

$$f_{jk}^p = q_{k-1}^p \prod_{l=1}^{k-1} r_{b_{jl}}^p (1 - r_{b_{jk}}^p), \quad \forall j \in J, k = 1, \dots, s, p \in \{H, L\}, \quad (1)$$

for all $j \in J$ and $k = 1, \dots, s$. Therefore, the correction factors restore the true dispatch probability f_{jk}^p when multiplied by the product of independent server busy probabilities.

We begin by providing the closed-form expression for the correction factors as

$$q_k^H = \frac{\sum_{i=k}^{s-1} \frac{(s-k-1)!(s-i)!}{(i-k)!s!} P_i}{v^k \rho^k (1 - v\rho)}, \quad (2)$$

$$q_k^L = \frac{\sum_{i=k}^{s_C-1} \frac{(s-k-1)!(s-i)!}{(i-k)!s!} P_i}{v^k \rho^k (1 - v\rho)}, \quad (3)$$

for every $k = 0, \dots, s-1$, in which

$$v = \begin{cases} \left(1 - P_s \frac{\lambda^H}{\lambda} - \sum_{i=s_C}^s P_i \frac{\lambda^L}{\lambda}\right) & \text{for the Loss model} \\ 1 & \text{for the Queued model} \end{cases} \quad (4)$$

where v refers to the probability that a call is immediately served. The derivation of q_k^p is detailed in Appendix A.

To compute these correction factors, we need to obtain the steady-state probabilities P_i , the probability that i servers are busy, $i = 0, \dots, s$, which also are performance measures of interest in this Hypercube queueing model. In terms of describing the steady-state probabilities, the state

Table 1 Summary of the notation

symbol	description
$j \in J$	set of call source nodes
$i \in I$	set of potential stations (used in the MILP)
$I^{open} \subset I$	set of open stations (used in the Hypercube model)
$p \in \{H, L\}$	set of call priorities
s	total number of servers
s_C	cutoff level for low priority calls
s_R	number of reserved servers, $s_R = s - s_C$
r_i	steady-state busy probability of server at station $i \in I$
r	steady-state system-wide average server busy probability, $r = \sum_i r_i / s$
P_i	steady-state probability that $i \in I$ servers are busy
q_k^p	correction factor for sending the k th preferred server, $k = 0, \dots, s - 1$, to priority $p \in \{H, L\}$ call
f_{jk}^p	probability of dispatching k th preferred server, $k = 1, \dots, s$ to priority $p \in \{H, L\}$ call from node $j \in J$
f_k^p	system-wide average probability that the k th preferred server is dispatched to a priority p call
$a_{ij}^p \in \{1, \dots, k\}$	preference order of server $i \in I$ by a priority $p \in \{H, L\}$ call from node $j \in J$
$b_{jk}^p \in I^{open}$	the station where the k th preferred server by priority $p \in \{H, L\}$ call from node $j \in J$ is located
λ_j^p	arrival rate of priority $p \in \{H, L\}$ call from node $j \in J$
$\lambda^H(\lambda^L)$	arrival rate of high(low) priority calls
λ	system-wide average call arrival rate, $\lambda = \lambda^H + \lambda^L = \sum_{j \in J} \sum_{p \in \{H, L\}} \lambda_j^p$
τ_{ij}	mean service time for calls from node $j \in J$ served by a server from station $i \in I$
τ	system-wide average mean service time. The initial value is given as $\tau^0 = \sum_{j \in J} \sum_{i \in I^{open}} \sum_{p \in \{H, L\}} \tau_{ij} (\lambda_j^p / \lambda^p)$, and this value is updated later in the algorithm
ρ	server utilization factor, $\rho = \lambda \tau / s$
w	relative weight of the low priority call coverage to high priority call coverage in the objective function
R_{ij}	probability that an ambulance located at station $i \in I$ successfully reaches the call from node $j \in J$ in a prespecified time limit
N_{ij}	set of call nodes that are neighbors to node j and are closer to station i than j
ϵ	server busy probability convergence threshold (used in the Hypercube model)
δ	imbalance measures that equals to standard deviation of server busy probabilities
γ	server workload imbalance threshold (used in the MILP)

is defined as the number of busy servers in the system. For the Loss model, we derive the closed-form expressions as follows.

$$P_i = \begin{cases} P_0 \frac{s^i \rho^i}{i!} & 0 \leq i \leq s_C - 1 \\ P_0 \frac{s^i \rho^i}{i!} \left(\frac{\lambda^H}{\lambda}\right)^{i-s_C} & s_C \leq i \leq s \end{cases} \quad (5)$$

where

$$P_0 = \left(\sum_{i=0}^{s_C-1} \frac{s^i \rho^i}{i!} + \sum_{i=s_C}^s \frac{s^i \rho^i}{i!} \left(\frac{\lambda^H}{\lambda}\right)^{i-s_C} \right)^{-1}. \quad (6)$$

For the Queued model, we refer the reader to Taylor and Templeton's work on cutoff priority queues [37] for the

detail of the derivation. They solve a series of balance equations to obtain the steady-state probabilities. Our Queued model refers to their SCQQ model so that both high and low priority calls are put in an infinite capacity queue when it is

not possible to immediately dispatch a server. We denote $\xi_1 = \lambda^H \tau$, $\xi_2 = \lambda^L \tau$, and $\xi = \lambda \tau$, where τ is the system-wide mean service time. Then the steady-state probabilities are

$$P_i = \begin{cases} P_0 \frac{\xi^s}{s!} & 0 \leq i \leq s_C - 1 \\ P_0 (\xi^{s_C} \xi_1^{i-s_C} / i!) s_C / (s_C - \xi_2 S_Q(s_C, s)) & s_C \leq i \leq s - 1 \\ P_0 (\xi^{s_C} \xi_1^{i-s_C} / i!) [s_C / (s_C - \xi_2 S_Q(s_C, s))] s / (s - \xi_1) & i = s \end{cases} \quad (7)$$

where

$$P_0 = \left(\sum_{i=0}^{s_C-1} \xi^i / i! + (\xi^{s_C} / (s_C - 1!)) S_Q(s_C, s) / [s_C - \xi_2 S_Q(s_C, s)] \right)^{-1}, \quad (8)$$

$$S_Q(k, s) = \xi_1^{-k} k! \left[\sum_{i=k}^{s-1} \xi_1^i / i! + (\xi_1^s / s!) s / (s - \xi_1) \right]. \quad (9)$$

In addition to the steady-state probabilities, we also evaluate the server busy probabilities r_i to obtain all the terms needed to compute the correction factors in Eqs. 2 and 3. As in other approximate Hypercube models [21, 26], we first obtain the expressions for server busy probabilities by applying Little's Law, as

$$r_i = \sum_p \sum_j \lambda_j^p \tau_{ij} f_{ja_{ij}^p}^p = \sum_p \sum_j \lambda_j^p \tau_{ij} q_{a_{ij}^p-1}^p \prod_{l=1}^{a_{ij}^p-1} r_{b_{jl}^p}^p (1 - r_{b_{jl}^p}^p), \quad \forall i \in I^{open}, \quad (10)$$

where $a_{ij}^p = k$ refers that i is the k th preferred server by priority p call from node j . Then we rearrange the above equalities to write

$$r_i = \frac{V_i}{V_i + 1}, \quad \forall i \in I^{open}, \quad (11)$$

where

$$V_i = \sum_{i \in I^{open}} \sum_{k \in K} \sum_{p \in \{H, L\}} \lambda_j^p \tau_{bjk}^p q_{k-1}^p (\prod_{l=1}^{k-1} r_{b_{jl}^p}^p), \quad \forall i \in I^{open}. \quad (12)$$

We can either compute the dispatch probabilities f_{jk}^p using individual server busy probabilities r_i , $i = 1, \dots, s$ as in Eq. 1, or approximate the same quantity using the system-wide busy probability $r = \sum_i r_i / s$, as $f_{jk}^p \approx q_{k-1}^p (1 - r) r^{k-1}$. This latter approximation becomes more precise when the individual server busy probabilities are close in magnitude, which occurs when the workload is balanced among the servers. In Section 4, we bound the server workload imbalance to be no more than 2% between ambulances in the computational study, which justifies the use of this approximation.

In a manner that is similar to [21], we use the above quantities to update system-wide service time τ as

$$\tau = \sum_{i \in I^{open}} \sum_{j \in J} \tau_{ij} \left(\frac{\lambda_j^H}{\lambda} f_{ja_{ij}^H}^H (1 - P_s) + \frac{\lambda_j^L}{\lambda} f_{ja_{ij}^L}^L (1 - \sum_{i=s_C}^s P_i) \right). \quad (13)$$

This update of τ is necessary as we have updated dispatch probabilities.

Finally, we summarize the iterative procedure for approximate Hypercube model with a cutoff priority queue, including the initialization step and termination criteria.

STEP 0 Given call arrival rates λ_j^p , service times τ_{ij} from inputs, open stations I^{open} and preference list b_{jk} , and the server busy probability convergence threshold ϵ , initialize system-wide service time τ and utilization factor ρ as

$$\tau^0 = \sum_{j \in J} \sum_{i \in I^{open}} \sum_{p \in \{H, L\}} \tau_{ij} \frac{\lambda_j^p}{\lambda^p}, \quad (14)$$

$$\rho = \frac{\lambda \tau}{s}. \quad (15)$$

STEP 1 Update steady-state probabilities P_i using Eqs. 5 and 6 for the Loss model, or using Eqs. 7, 8, and 9 for the Queued model.

STEP 2 Update the correction factors q_k^p using Eqs. 2 and 3, where v follows (4) for the Loss model or $v = 1$ for the Queued model.

STEP 3 Update the server busy probabilities r_i using Eqs. 11 and 12.

STEP 4 Update the dispatch probabilities f_{jk}^p using Eq. 1, and normalize them by

$$f_{jk}^H \leftarrow \frac{f_{jk}^H (1 - P_s)}{\sum_{k \in K} f_{jk}^H}, \quad \forall j \in J, k = 1, \dots, s, \quad (16)$$

$$f_{jk}^L \leftarrow \frac{f_{jk}^L (1 - \sum_{i=s_C}^s P_i)}{\sum_{k \in K} f_{jk}^L}, \quad \forall j \in J, k = 1, \dots, s. \quad (17)$$

Update the system-wide service time τ using Eq. 13 with the updated dispatch probabilities.

STEP 5 Update the utilization factor $\rho = \frac{\lambda\tau}{s}$ and system-wide busy probability $r = \nu\rho$, where ν has the same value as STEP 2. Normalize the busy probabilities by

$$r_i \leftarrow \frac{r_i}{\sum_{l \in I^{open}} r_l / s} r, \quad \forall i \in I^{open} \quad (18)$$

Check if the maximum change in r_i between iterations is smaller than the server busy probability convergence threshold ϵ . If yes, terminate and return r_i for $i \in I^{open}$, q_k^p (and P_i for $i = 0, \dots, s$, if the Queued model is used). If not, go back to STEP 1.

There are no proven theoretical results or bounds that guarantee the convergence of this procedure in general without further assumptions. However, our numerical experiments have demonstrated a fast convergence in a small number of iterations when the cutoff is not too low. Other Hypercube models also report fast convergence in their numerical studies [21, 26].

Note that the model described in this section is an approximate model. Therefore, we demonstrate its accuracy in Section 4 by comparing the Hypercube model approximations to the results of a discrete event simulation in a computational study.

3 Maximum expected coverage model

In this section we present a mixed integer linear program model that simultaneously locates servers at stations and dispatches servers to call nodes. The goal of the model is to maximize the expected coverage over the region. A call is considered to be covered if a server is dispatched to the call immediately upon the arrival and it reaches the call in the predetermined time limit (e.g., 9 minutes). The expected coverage is defined as the average proportion of calls successfully covered given the deployment and dispatching policy. The expected coverage is weighted over high priority and low priority calls, and it uses the busy probabilities and correction factors from the the approximate Hypercube model. The MILP model distinguishes between different call priorities in determining ambulance location and dispatch policies, and it allows for a cutoff priority scheme. The MILP model adapts the ambulance location and dispatch model of Ansari et al. [1] and extended to involve cutoff priority scheme.

The inputs to the MILP model are busy probabilities and correction factors from the approximate Hypercube model. The MILP solution outputs are the set of open stations and the preference lists for each call priority, which then become

inputs for the Hypercube model. Therefore, we alternatively solve the approximate Hypercube model described in the previous section and the MILP model, since each model's outputs provides the inputs to the other model. At the end of this section, we provide a Hypercube-MILP iterative procedure where the two models alternatively run, thus updating the inputs for each model until the server workloads are approximately balanced.

Both the Loss and the Queued models use the same MILP model illustrated in this section, except for one set of constraints that enforce load balancing among the servers. The Hypercube model returns different values of correction factors and server busy probabilities for various sizes of the queue, and these values are incorporated into the MILP model to generate deployment and dispatch decisions optimized for the type of the model.

We select s stations from a set of I potential stations for locating servers. The binary decision variables capture the public safety system design decisions. Let $y_i = 1(0)$ if a server is (not) located at station $i \in I$. This variable set defines the set of open stations. Let $x_{ijk}^p = 1(0)$, $i \in I$, $j \in J$, $k = 1, \dots, s$ if station i is (not) the k th preferred station for a priority p call from node j . This variable set captures the dispatch policy for all types of calls. Note that the resulted optimal dispatch plan may deviate from the natural sending-the-closest-idle policy (see also [29] and [19]).

The mixed integer linear program is formally stated below.

$$\max_x \sum_{i \in I} \sum_{j \in J} \sum_{k=1}^s (c_{ijk}^H x_{ijk}^H + w c_{ijk}^L x_{ijk}^L) \quad (19)$$

subject to

$$\sum_{i \in I} y_i = s, \quad (20)$$

$$\sum_{k=1}^s x_{ijk}^p = y_i, \quad \forall i \in I, j \in J, p \in \{H, L\}, \quad (21)$$

$$\sum_{i \in I} x_{ijk}^p = 1, \quad \forall j \in J, k = 1, \dots, s, p \in \{H, L\}, \quad (22)$$

$$\sum_{j \in J} \sum_{k=1}^s \sum_{p \in \{H, L\}} \lambda_j^p q_{k-1}^p (1-r)^{k-1} \tau_{ij} x_{ijk}^p \leq (r+\delta) y_i, \quad \forall i \in I, \quad (23)$$

$$\sum_{j \in J} \sum_{k=1}^s \sum_{p \in \{H, L\}} \lambda_j^p q_{k-1}^p (1-r)^{k-1} \tau_{ij} x_{ijk}^p \geq (r-\delta) y_i, \quad \forall i \in I, \quad (24)$$

$$x_{ij'1}^H \geq x_{ij1}^H, \quad j \in J, i \in I, j' \in N_{ij}, \quad (25)$$

$$y_i \in \{0, 1\}, \quad i \in I, \quad (26)$$

$$x_{ijk}^p \in \{0, 1\}, \quad i \in I, j \in J, k = 1, \dots, s, p \in \{H, L\}. \quad (27)$$

The objective function in Eq. 19 represents the weighted expected coverage over high priority calls and low priority calls. The weight parameter w reflects the relative significance of low priority calls in the decision maker's objective compared to high priority calls (which have an implicit weight of 1). Without loss of generality we assume $w \in [0, 1)$ as it is more important to cover time-sensitive high priority calls. The coefficient c_{ijk}^p used in the objective function is computed as

$$c_{ijk}^p = q_{k-1}^p (1-r) r^{k-1} R_{ij} \frac{\lambda_j^p}{\lambda^H + w\lambda^L}, \quad \forall i \in I, j \in J, k = 1, \dots, s, p \in \{H, L\}. \quad (28)$$

This represents the weighted fraction of calls of priority p at node j that are reached in the prespecified time limit by server i if it is the k th preferred server. The objective coefficient (28) can be divided into three components. The first component $q_{k-1}^p r^{k-1}$ is the probability that the k th preferred server is dispatched to a priority p call. This component captures ambulance unavailability and uses the system-wide server busy probability r and the correction factor q_{k-1}^p , which are the output from approximate Hypercube model. This yields an approximation to the queueing probabilities that compensates for the independence assumption between the servers. The second component R_{ij} , which captures travel time uncertainty, represents the probability that an ambulance located at station i reaches a call at node j within the specified time limit [18]. The first two components together ($q_{k-1}^p (1-r) r^{k-1} R_{ij}$) capture the fraction of calls at node j covered by station i when i is the k th preferred server for calls at node j . The final component weighs calls by the demand (call volume) and the priority and normalizes the weights, thus yielding a weight of $\lambda_j^p / (\lambda^H + w\lambda^L)$ for priority $p \in \{H, L\}$ calls at node $j \in J$.

The cutoff priority queue scheme is captured in the MILP model through server busy probabilities and the correction factors. To capture calls being cut off, the low priority correction factor q_{k-1}^L is set to zero for all $k > s_C$. Therefore, the objective coefficient $c_{ijk}^L = 0$ when $k > s_C$, since the cutoff priority queue discipline restricts the Hypercube model to always return $q_{k-1}^L = 0$ for $k > s_C$. As a result, x_{ijk}^L for $k > s_C$ is "inactive" in practice, since the cutoff prevents dispatching to low priority calls when k or more ambulances are busy. We keep the variables x_{ijk}^L for $k > s_C$ as it simplifies notation and does not affect the objective value or the feasibility set. Note that $\{x_{ijk}^p\}$ is a binary representation of the preference list $\{b_{jk}^p\}$. Once the MILP problem is solved, the preference lists for high and low-priority patients are set using the optimal MILP solution variables as $b_{jk}^p \leftarrow \{i | x_{ijk}^p = 1\}$, $j \in J, k = 1, \dots, s$ and $b_{jk}^L \leftarrow \{i | x_{ijk}^L = 1\}$, $j \in J, k = 1, \dots, s_C$.

We next describe the constraints in the model. A station is considered to be open if a server is located at it. Exactly s stations are open, which is guaranteed by Eq. 20. A server from station i is dispatched only when the station i is open, which is enforced by Eq. 21. Constraint set (22) requires every call node to have an ordered, non-overlapping preference list of available servers. Constraint sets (26) and (27) require the variables to be binary.

Constraint sets (23) and (24) are load balancing constraints that set upper and lower limits for server utilization. The server utilization for each open station is required to be within a tolerance δ of the average server utilization. These constraint sets have two implications. First, the program uses common busy probability r instead of r_i , $i \in I^{open}$ with an implicit assumption that the busy probability is the same for all servers. This assumption is reasonable, since (23) and (24) ensure that service providers spend approximately the same proportion of time serving calls, which is desirable for service providers maintaining their skill proficiency [29].

Constraint sets (23) and (24) are valid for the Loss model. For the Queued model, we replace (23) and (24) with the following constraints:

$$\sum_{j \in J} \sum_{k=1}^s \sum_{p \in \{H, L\}} \lambda_j^p q_{k-1}^p (1-r) r^{k-1} \tau_{ij} x_{ijk}^p \leq (r + \delta - \frac{\lambda \tau P_D}{s}) y_i, \quad \forall i \in I, \quad (29)$$

$$\sum_{j \in J} \sum_{k=1}^s \sum_{p \in \{H, L\}} \lambda_j^p q_{k-1}^p (1-r) r^{k-1} \tau_{ij} x_{ijk}^p \geq (r - \delta - \frac{\lambda \tau P_D}{s}) y_i, \quad \forall i \in I, \quad (30)$$

where the parameter $P_D = P_s \frac{\lambda^H}{\lambda} + \sum_{i=s_C}^s P_i \frac{\lambda^L}{\lambda}$ captures the probability of a call being delayed and put in an infinite capacity queue. The additional term $\frac{\lambda \tau P_D}{s}$ represents the workload of serving delayed calls in the queue. The steady-state probabilities P_i for $i = 0, \dots, s$ can be computed using the Hypercube model, along with correction factors and server busy probabilities.

Finally, constraint set (25) requires the first priority districts to be contiguous so that the resulting dispatching policy yields contiguous first priority response districts for each open station, and it results in contiguity as follows. When the first preferred station of high priority calls from node j is station i , then constraint set (25) requires a node j' that is a neighbor of j and closer to station i ($j' \in N_{ij}$) to also have its first preferred station for high priority calls to be i ($x_{ij'1}^H = 1$). This requirement yields a contiguous district surrounding i .

The system-wide server busy probability r is treated as an exogenously provided input parameter in the objective

coefficients as well as load balancing constraints. However, r is endogenous to the model in that it is a function of the decision of deployment and dispatching decisions. To maintain model tractability, a second iterative scheme is introduced. As mentioned in the previous section, the approximate Hypercube model generates the correction factors q_k^p and server busy probabilities r used as inputs for MILP model in the first iterative procedure. Then the solutions from the MILP model become new inputs for the Hypercube model that updates the value of correction factors and server busy probabilities. Again, those outputs are utilized as inputs for the MILP model. The second iterative procedure continues until the outputs converge, and server busy probabilities are approximately balanced. As this iterative procedure is an approximate algorithm, later we validate its accuracy through computational study in Section 4.

Below we detail the steps in iterative algorithm, which adopts the procedure introduced by [1] with a cutoff priority scheme. It is notable that this procedure is guaranteed to converge (termination criteria (1)) unless it leads to infeasibility (termination criteria (2)). In STEP 3, the imbalance parameter δ is updated with the standard deviation of r_i , $i \in I$, and it always decreases because its value from the previous iteration set upper and lower bounds for the server busy probabilities r_i , $i \in I$ through the load balancing constraint sets in MILP of STEP 1. Therefore, it is ensured that δ has shrinking margin every iteration, and the procedure terminates at the point that δ falls under the predetermined threshold value γ or it imposes a tight load balancing constraint that is too strong to be feasible (see [1]).

STEP 0 Initialize:

Generate the initial preference lists b_{jk}^p , by always sending the closest available server.

Set correction factors $q_k^p = 1$ for all $k = 1, \dots, s$, server busy probabilities $r = r_i = \rho = \frac{\lambda\tau}{s}$ for all $i \in I^{open}$, where the system-wide mean service time τ is set to $\tau = \lambda^{-1} \sum_{i \in I} \sum_{j \in J} \sum_{p \in \{H, L\}} \lambda_j^p \tau_{ij}$. Set the imbalance tolerance to $\delta = 1.0$.

STEP 1 Solve the MILP model with inputs r , q_k^p (and P_D if the Queued model is used) and update preference lists b_{jk}^p and set of open stations I^{open} .

STEP 2 Solve the approximate Hypercube model with inputs b_{jk}^p and I^{open} to update server busy probabilities r and correction factors q_k^p .

STEP 3 At the 1st iteration, update $\delta = r^{max} - r^{min}$ and go back to STEP 1. Otherwise, update the imbalance parameter δ with the standard deviation of r_i , $i \in I^{open}$. Check termination criteria: stop if (1) the imbalance measure δ is less than the prespecified server workload imbalance threshold γ or if (2) MILP was infeasible. Else if both termination criteria are not satisfied, go back to STEP 1.

4 Computational results

This section reports analytical and discrete event simulation results that validate the models and illustrate the effectiveness of introducing the cutoff priority queue scheme for emergency medical service systems. While we mainly consider the Loss model, we also report computational results for the Queued model for comparison. The validation for how accurately the proposed Hypercube model approximates system performance measures of the system with a cutoff is provided through comparison to simulation results. Throughout this section we use $s_R = s - s_C$, the number of servers reserved exclusively for high priority calls, in describing the cutoff priority scheme.

4.1 Computational setting

The model is studied using real-world data set from Hanover County, Virginia [30]. The data describes a geographical region with 16 potential stations, 270 aggregated demand nodes with 139 2×2 mile cells and 131 1×1 mile cells. The calls during the weekdays from 12PM to 6PM over 31 months is used to estimate the call arrival rates (demand). The call arrival rate is approximately constant over this time period. The demand over the region is visualized in Fig. 2 as a colored cells on a grid, where a darker color represents a higher arrival rate. The arrival rate over the area is not uniform, as it is concentrated in the middle and south-east areas of the county while the arrival rates are low on the west side. The location of 16 potential stations are also marked in the map. The base case considers $s = 5$ servers.

To investigate the effect of reserving capacities in a large fleet system, we also consider a “scaled-up” version of the Hanover County dataset with a larger fleet. To generate the scaled-up dataset, the number of servers increases from 5 to 16, and the call arrival rates is scaled by a factor of 5.9. The

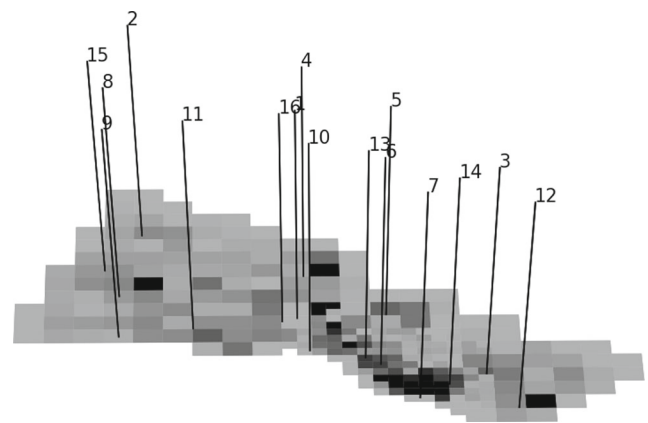


Fig. 2 Demand Distribution and Station Location on the Hanover County Map

Table 2 High Priority Dispatch Probabilities f_k^H and Loss Probabilities for High Priority Calls $P(\text{lost}|H)$ for Different Values of s_R

	Hypercube					Simulation				
	$s_R = 0$	$s_R = 1$	$s_R = 2$	$s_R = 3$	$s_R = 4$	$s_R = 0$	$s_R = 1$	$s_R = 2$	$s_R = 3$	$s_R = 4$
$k = 1$	0.6669	0.6770	0.6961	0.7326	0.7857	0.6594	0.6700	0.6903	0.7272	0.7813
$k = 2$	0.2012	0.2013	0.2040	0.1978	0.1724	0.1980	0.1990	0.2007	0.1931	0.1664
$k = 3$	0.0696	0.0703	0.0664	0.0520	0.0316	0.0732	0.0733	0.0695	0.0568	0.0379
$k = 4$	0.0280	0.0279	0.0219	0.0116	0.0068	0.0316	0.0309	0.0253	0.0153	0.0099
$k = 5$	0.0131	0.0135	0.0065	0.0035	0.0020	0.0150	0.0162	0.0088	0.0049	0.0029
$P(\text{lost} H)$	0.0212	0.0100	0.0050	0.0026	0.0016	0.0227	0.0106	0.0053	0.0027	0.0016

demand scale parameter 5.9 is selected so that the system shows the similar coverage performance to the original 5-server case when there is no cutoff ($s_C = s$). Note that we do not choose the scale to fix the ratio between the demand volume and the number of servers. As a result, the server utilization factor of the large-scale dataset is 1.84 times greater than the original dataset. This large-scale dataset represents a higher traffic system with more resources compared to the original dataset. The effect of the reserved capacity on coverage appears different in low traffic and high traffic systems. By choosing the scale parameter that lines up the performance of baseline models where there is no cutoff, we enable a direct comparison of the coverage increment trend between the two systems. Importantly, this choice of a scale parameter results in a traffic intensity that employs the cut-off under a variety of settings and allows us to study a range of values for the cutoff.

The Hypercube model and the MILP model in the iterative procedure were implemented in Python 2.7.8 and solved using Gurobi 7.0.1. The running time of the Loss model for the iterative MILP-Hypercube procedure using Hanover County data is 27.13 seconds on average, and the time required for running the Hypercube models adds less than 0.11 seconds for all cases. The runtime for the scaled-up data with a large fleet is 40.87 seconds on average, and the Hypercube model runtime always adds less than 0.5 seconds. The load imbalance tolerance threshold γ in the termination criteria of the iterative procedure is set to 2% for the original dataset and 15% for the scaled dataset, and

server busy probability convergence threshold ϵ is set to 1% for both. The computational time performance and parameter settings for the Queued model is analogous to the Loss model.

4.2 Model validation

We compare analytical results obtained by the approximate Hypercube model to discrete-event simulation results using the Loss model to demonstrate the accuracy of the approximate Hypercube model. The simulation uses the same demand distribution and geographical data as the analytical model, and the resulting deployment and dispatch decisions from the analytical model are also used as policy inputs for the simulation. Note that unlike the Hypercube model and the MILP model that are iterative, the simulation only receives deployment and dispatch decisions from the MILP model after convergence, and the simulation output is compared to the Hypercube model output without further iteration. The program was implemented in Python 2.7.8, ran until 100,000 calls arrive, and repeated 30 times to generate summary statistics for dispatch probabilities, server busy probabilities, steady-state probabilities and coverage over the high priority calls. Since the system of concern is relatively low-traffic, and the duration of the simulation is long enough to ignore the transient behavior at the beginning of the simulation, we do not incorporate a warm-up in our simulation for the original dataset. For the scaled-up version with higher traffic, we use a warm-up period of 2,000 calls before starting to collect results.

Table 3 Low Priority Dispatch Probabilities f_k^L and Loss Probabilities for Low Priority Calls $P(\text{lost}|L)$ for Different Values of s_R

	Hypercube					Simulation				
	$s_R = 0$	$s_R = 1$	$s_R = 2$	$s_R = 3$	$s_R = 4$	$s_R = 0$	$s_R = 1$	$s_R = 2$	$s_R = 3$	$s_R = 4$
$k = 1$	0.6709	0.6658	0.6293	0.5154	0.2798	0.6655	0.6583	0.6222	0.5072	0.2742
$k = 2$	0.1987	0.1889	0.1487	0.0729	0.0000	0.1994	0.1896	0.1503	0.0732	0.0000
$k = 3$	0.0689	0.0570	0.0281	0.0000	0.0000	0.0708	0.0582	0.0286	0.0000	0.0000
$k = 4$	0.0275	0.0153	0.0000	0.0000	0.0000	0.0284	0.0162	0.0000	0.0000	0.0000
$k = 5$	0.0127	0.0000	0.0000	0.0000	0.0000	0.0133	0.0000	0.0000	0.0000	0.0000
$P(\text{lost} L)$	0.0212	0.0731	0.1939	0.4117	0.7202	0.0226	0.0777	0.1990	0.4196	0.7258

Table 4 System-wide Server Busy Probability r for Different Values of s_R

	Hypercube					Simulation				
	$s_R = 0$	$s_R = 1$	$s_R = 2$	$s_R = 3$	$s_R = 4$	$s_R = 0$	$s_R = 1$	$s_R = 2$	$s_R = 3$	$s_R = 4$
r	0.3303	0.3216	0.3026	0.2628	0.2085	0.3367	0.3281	0.3066	0.2668	0.2115

Tables 2 and 3 display dispatch probabilities for the original Hanover County dataset with 5 servers, for different values of reserved servers s_R . System-wide dispatch probabilities f_k^p provided in the table are computed using the dispatch probabilities from the Hypercube model as

$$f_k^p = \sum_{j \in J} \frac{\lambda_j^p}{\lambda^p} f_{jk}^p, \quad \forall p \in \{H, L\}, k = 1, \dots, s. \quad (31)$$

The left hand columns of Tables 2 and 3 report the analytical dispatch probabilities from the approximate Hypercube model, while the right hand columns report the average simulation dispatch probabilities. Table 4 reports the server busy probabilities for the original dataset in both the approximate Hypercube model and the simulation model. Dispatch probabilities are all zero for $k > s_C$ for low priority calls, since low priority correction factors q_{k-1}^L from Eq. 3 are all zero for $k > s_C$, and they are entered into Eq. 1 to yield dispatch probabilities of zero. This is the direct result of cutoff priority queue discipline. As we lower the cutoff s_C , the probability of dispatching the first preferred server to high priority calls increases.

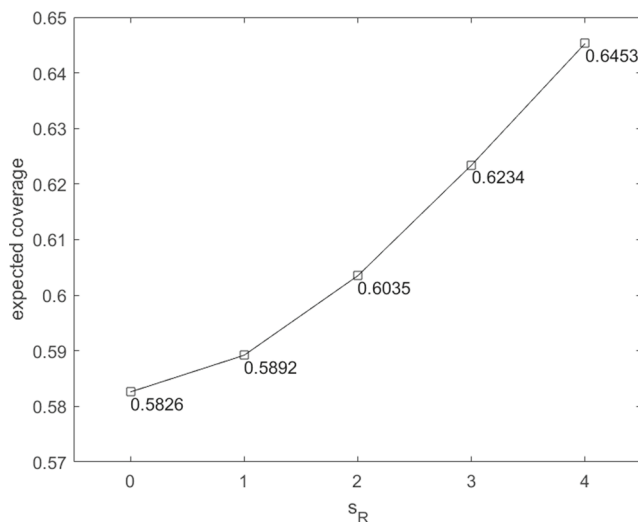
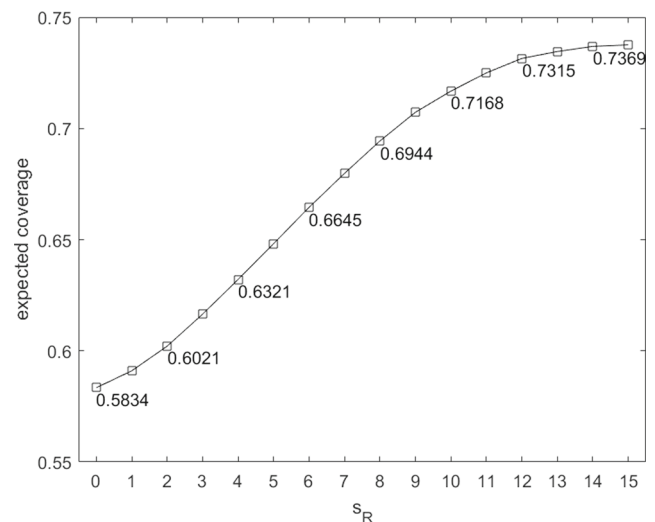
Tables 2–4 enable us to compare various outputs from the approximate Hypercube model to their corresponding values from the simulation. The tables report that the discrepancy between the analytical Hypercube model results and simulation results are small. The absolute errors for the server busy probability r in Table 4 are less than 0.65%

across all values of s_R . The dispatch probabilities in first five rows in Tables 2 and 3 are also very close to the simulation results with the absolute error of at most 0.64%. Finally, the numbers in the last row of Table 2 refer to the loss probabilities, which are estimated with the absolute error of no more than 0.79%. The tables corresponding to Tables 2–4 for the Scaled-up Dataset are presented in Appendix B.

The computational time for the simulation average 479.60 seconds for the original dataset and 525.49 seconds for the scaled-up dataset. Note that the runtime of the simulation is significantly longer than the runtime of Hypercube model, which was less than 0.11 seconds for the original dataset and 0.50 seconds for the scaled-up dataset, while they both serve the function of evaluating the system given preference lists. We note that some output statistics such as dispatch probabilities to less preferred ambulances can be relatively rare events and the simulation should be run for as long as 100,000 calls in our example, and possibly longer when there are more ambulances in the system. This highlights a benefit of an analytical model such as the Hypercube model, whose computational time scales well in the number of servers.

4.3 System design and coverage improvement

Here we consider the Loss model and set $w = 0$ so that the MILP objective is to maximize the expected coverage

**Fig. 3** Expected Coverage for Different Values of s_R in the Hanover County dataset with $s = 5$, $w = 0$ **Fig. 4** Expected Coverage for Different Values of s_R in the Scaled-up Hanover County dataset with $s = 16$, $w = 0$

over high priority calls. This allows us to demonstrate the impact of the cutoff on high-priority calls. Figure 3 shows the expected coverage over high priority calls from the MILP model with the original dataset, and Fig. 4 shows the expected coverage over high priority calls for the scaled-up version of Hanover County dataset with 16 servers.

The effect of the reserved capacity on high-priority call coverage is more pronounced in the scaled-up example, as illustrated in Fig. 4. Specifically, the marginal improvement in expected coverage is 1.63% when the number of reserved servers s_R increases from five to six. Given that the maximum achievable coverage of this system is only 81.28% that occurs under the impractical assumption that every call is served immediately by the server located at the most reachable station, the improvement from 64.82% to 66.45% by reserving one additional server is substantial.

The expected coverage is monotonically increasing in the number of reserved servers s_R . The expected coverage improves as more servers are reserved based on a combination of two factors. First, as more servers are reserved for high priority call arrivals, servers are less busy (see Table 4). Second, as more servers are reserved for high priority call arrivals, the system is more likely to send highly preferred servers that are nearby and is therefore more likely to “cover” the calls. This can be seen in the left half columns of Tables 2 and 3 for the original Hanover County dataset with $s = 5$ servers. The first and second choice servers are more likely to be sent—as shown by dispatch probabilities with $k = 1, 2$ for high priority calls—when s_R increases.

However, the improvement in expected coverage for high priority calls comes at the cost of losing more low priority calls. As seen in Table 3, in the base case of reserving no servers, the probability of losing a low priority call is 2.12%, and it increases to 7.31% when reserving one server, while resulting a coverage improvement of 0.66%. Therefore, there is a tradeoff between improving high priority coverage by reserving servers and staying responsive to low priority calls. It is up to decision makers to choose the level of cutoff with allowable loss for low priority calls.

The cutoff priority scheme leads to different design decisions in the MILP model. Figure 5 shows the optimal district designs for the scaled-up Hanover County dataset

with sixteen servers. Each color block with a station number represents a first priority district covered by the station. The districts associated with stations 6, 7, 13, and 14 that are the most populated areas in call arrivals, show the greatest changes when cutoff is modified. Therefore, the cutoff leads to different district designs for high priority calls, which contributes to a greater increase in the expected coverage than by solely reserving capacities. This suggests that planning decisions should be altered in response to the introduction of a cutoff, which is an important insight.

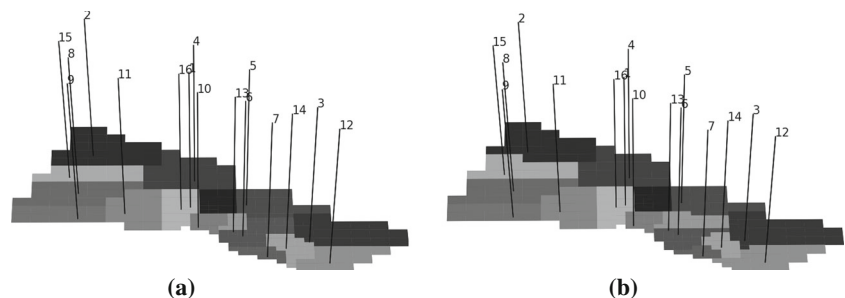
4.4 Identifying the number of reserved servers

Thus far, we discuss how to improve coverage for high priority calls at the expense of losing more low priority calls by introducing a low priority cutoff level s_C and setting the low priority weight parameter $w = 0$. In this section, we consider $w > 0$ to explicitly consider the effect of losing low priority calls in the objective function. Setting $w = 0$ means we only consider high priority coverage in evaluating the performance of the system, while $w = 1$ implies that high and low priority calls are equally valued. Relatively smaller values such as $0.1 \leq w \leq 0.5$ are more realistic, since emergency calls with higher priorities are more time-sensitive and have a higher necessity to be served in a timely manner (i.e., covered).

Figures 6 and 7 show the expected weighted coverage for different cutoffs and weights in the original and scaled-up Hanover County dataset. These curves are no longer monotonically increasing in the number of reserved servers as earlier with $w = 0$, since the lost low-priority calls affect the expected weighted coverage here. Hence for each $w > 0$, there exist different values of s_R (with $s_R \leq s$) that maximize total expected coverage, and this analysis can inform decision makers regarding how to select the number of servers to reserve.

Consider the $w = 0.1$ case for the original dataset in Fig. 6. From Fig. 6 we can see that the expected coverage reaches its maximal value at $s_R = 3$, and starts to decrease when s_R increases beyond 3. Therefore, we can conclude that reserving $s_R = 3$ servers yields the highest expected weighted coverage and reserving more than three servers

Fig. 5 First-priority Districts for High Priority Calls with (a) $s_R = 8$ and (b) $s_R = 10$ in the Scaled-up Hanover County Dataset with $s = 16$, $w = 0$



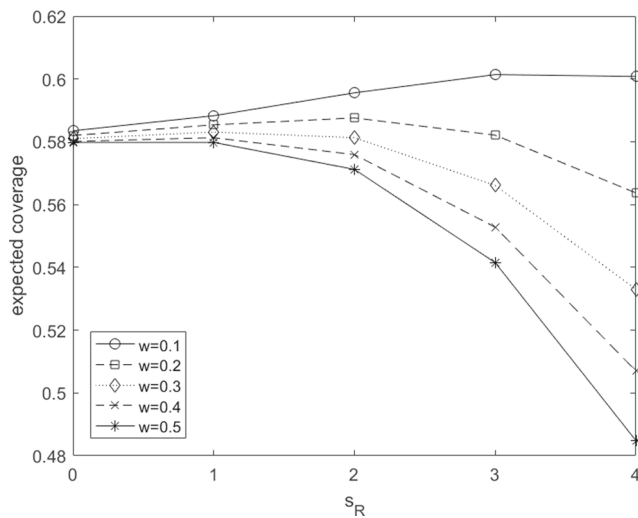


Fig. 6 Expected Coverage with Different Weight on Low Priority Calls in Hanover County Dataset with $s = 5$

is suboptimal. As the weight w increases, the maximal expected weighted coverage for each value of w occurs by reserving fewer servers. Specifically when w is as large as 0.5 so that low priority calls are valued half as important as high priority calls in the objective function, expected coverage reaches its maximal values when no servers are reserved ($s_R = 0$). Similarly, for the Scaled-up dataset as seen in Fig. 7, the value of s_R that yields the highest expected coverage decreases from 11 to 5 as we increase the weight on low priority calls from 0.1 to 0.5. Therefore, a cutoff priority scheme is more effective in improving the expected total coverage when high priority calls are weighed significantly more than low priority calls.

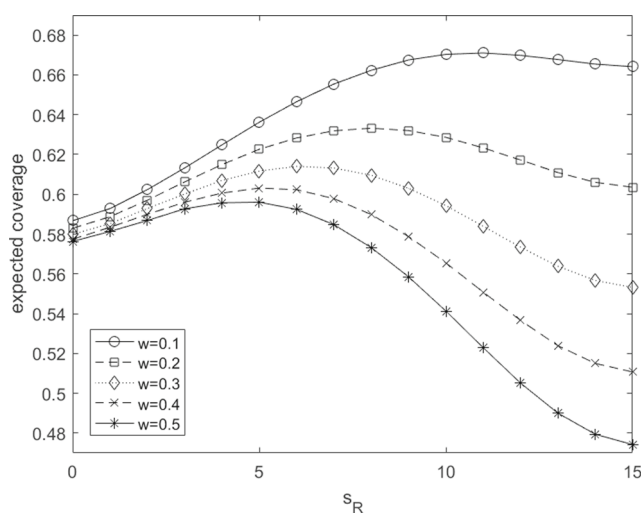


Fig. 7 Expected Coverage with Different Weight on Low Priority Calls in the Scaled-up Hanover County Dataset with $s = 16$

4.5 The system with the infinite capacity queue

In this section we report results from the system with infinite-capacity queue, to understand how total expected coverage can be improved by reserving capacity when we maintain a queue and calls are not lost.

Figure 8 illustrates the expected coverage for the Queued and the Loss models as a function of s_R . The expected coverage in the Queued model is lower than in the Loss model. With an infinite capacity queue, servers ultimately serve every call, and therefore, the busy probabilities and dispatch probabilities are similar regardless of how many servers are reserved (see Table 5), since there is still a similar amount of “work” in the system. As servers become busy serving low priority calls that have been delayed service by the cutoff, there is smaller benefit of improving the total expected coverage from reserving capacity.

By maintaining an infinite capacity queue, low priority calls are never abandoned. The Queued model can become unstable and the low priority call queue can become infinitely long if too many servers are reserved, which occurs for this example when $s_R = 4$ ($s_C = 1$) and the realized service rate for low-priority calls is as low as $P_0\mu$. Compared to the Loss model, the Queued model provides an useful tool for estimating system performances when calls are not abandoned or resorted to external services, as long as the resulting system is stable.

Lastly, a mixed situation where the calls are entered into a queue probabilistically can be considered. This implementation is applicable when external resources such as mutual aid is not always available. It is expected that the performance of a mixed system when the cutoff discipline

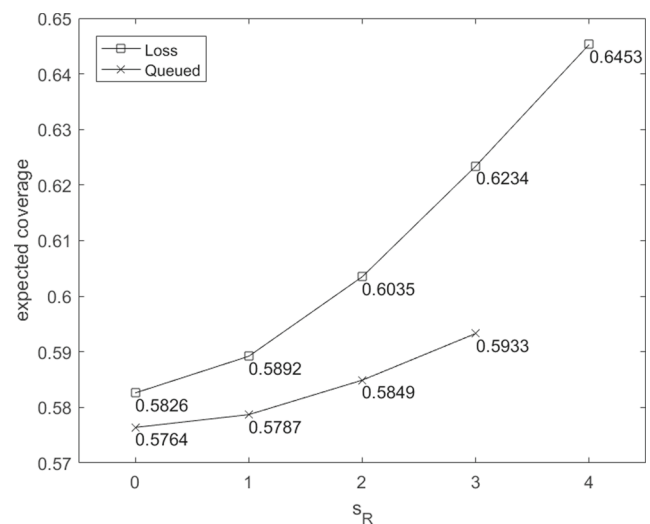


Fig. 8 Expected Weighted Coverage with an Infinite Capacity Queue and $s = 5$, $w = 0$. *Result for $s_R = 4$ is omitted due to infeasibility

Table 5 Dispatch Probabilities for Different Values of s_R With Infinite-capacity Queue and $s = 5$

	High Priority				Low Priority			
	$s_R = 0$	$s_R = 1$	$s_R = 2$	$s_R = 3$	$s_R = 0$	$s_R = 1$	$s_R = 2$	$s_R = 3$
k=1	0.6621	0.6615	0.6617	0.6553	0.6624	0.6451	0.5610	0.3198
k=2	0.1988	0.2018	0.2126	0.2368	0.1978	0.1838	0.1354	0.0457
k=3	0.0680	0.0719	0.0785	0.0807	0.0683	0.0560	0.0255	0.0000
k=4	0.0269	0.0318	0.0297	0.0174	0.0273	0.0147	0.0000	0.0000
k=5	0.0122	0.0171	0.0086	0.0052	0.0125	0.0000	0.0000	0.0000
P(delayed)	0.0317	0.0160	0.0084	0.0048	0.0317	0.1003	0.2768	0.6354

Result for $s_R = 4$ is omitted, because if we solve the model with infinite capacity queue using this dataset, the system is unstable

is adapted to this case lies in between the two curves in Fig. 8. Intuitively the result would be closer to the Loss model when external resources are available with higher probability.

5 Conclusion

This paper introduces models to evaluate the design of public safety systems with a cutoff priority queue, and it provides insights into the benefits that can be achieved from reserving some capacity for high priority calls. We introduce an approximate Hypercube model with cutoff priority queue, and we embed this model into a MILP model so we can identify how to locate and dispatch ambulances to respond to prioritized calls for service.

Our computational analysis shows that the use of iterative MILP-Hypercube procedure is a computationally efficient way to model and solve the problem. The analytical model provides a relatively efficient approximation that requires less computational time compared to discrete event simulation and is also accurate. The computational study demonstrates an improvement in coverage over high priority calls when introducing a cutoff for low priority calls using a real-world dataset. This improvement in coverage for high priority calls is accentuated when we consider a scaled-up dataset with more servers and a higher demand.

There is a tradeoff between improving coverage for high priority calls at the expense of “losing” more low priority calls when a cutoff is introduced. We study this tradeoff by evaluating the expected weighted coverage over both low and high priority calls. The analysis sheds light on how to identify the “right” number of servers to reserve, which can aid decision makers in setting the cutoff.

We also model a system where low priority calls are not lost due to cutoff but are rather served later when there are enough free servers. A system with an infinite capacity queue (Queued model) is implemented and compared to the Loss model. The system with an infinite capacity queue

leads to a smaller improvement in coverage when more servers are reserved as compared to the system with a zero capacity queue. This is because low priority calls are delayed rather than lost when the server availability is below the cutoff, and therefore, ambulances are ultimately not “freed up” when there an infinite capacity queue. This suggests that a cutoff has the most potential to improve system performance when cutoff low-priority calls are ultimately serviced by external resources.

The model we provide can be extended. First, our framework can be applied to a system with more than two priority classes by imposing multiple cutoff numbers. This could be practical, since many emergency medical service systems are based on more than two patient priorities. Second, the relationship between the optimal cutoff and the system parameters such as call volume or fleet size can be further studied to understand how to reserve ambulances when there is a sudden increase in demand. The analysis could give insight into the development of emergency plans for mass casualty events and a surge in demand due to severe weather. Third, although 8- or 9-minutes are the most frequently used time thresholds when defining the call coverage, longer thresholds are often used for low priority patients or in rural settings. Our model can be applied to such EMS systems by replacing the reach parameters R_{ij} with priority-specific reach parameters, R_{ij}^p , $p \in \{H, L\}$. Finally, we use simulation to demonstrate the accuracy of the models. The framework developed in this paper could be considered a “Solution-Completion by Simulation” (SCS) model, in which the solution obtained by the optimization is applied in a simulation environment to discover more details of the model [13].

Acknowledgements This work was funded by the National Science Foundation [Awards 1444219, 1361448]. The views and conclusions contained in this document are those of the author and should not be interpreted as necessarily representing the official policies, either expressed or implied, of the National Science Foundation. The authors would like to thank the anonymous reviewers for their valuable comments and suggestions to improve the quality of the paper.

Appendix A: Derivation of approximate hypercube model with cutoff priority queue

In this appendix, we present derivation details for the approximate Hypercube model for a cutoff priority queue presented in Section 2. First, we derive closed-form expressions for the steady-state probabilities in a cutoff priority queue. The steady-state probability formulations in Eqs. 5 and 6 are easily obtained from balance equations in the standard birth-death process. Figure 9 shows the corresponding transition diagram for a cutoff priority queue systems with a zero-line queue.

To derive the steady-state probabilities formulation for the infinite capacity queue in Eqs. 7, 8 and 9, we adapt the SCQQ model in [37], where both high and low priority calls are put in an infinite capacity queue when the number of busy servers is equal to or greater than the cutoff (s for high and s_C for low priority calls). In their paper they also provide the formula for the case where high priority customers are lost while only low priority customers are queued (SCQL model), which we do not consider. Therefore if one wants to apply the idea of reserving capacity only with a low priority queue, that model can be directly used in our procedure too, with updates for the steady-state probabilities P_i in STEP 1 and by setting $\nu = 1 - P_s \frac{\lambda^H}{\lambda}$ in STEP 2 (of the iterative Hypercube procedure in Section 2).

Next, we derive correction factors for low priority calls in the cutoff priority queue. This follows the procedure in [26] closely, with some modifications due to the cutoff assumption. Suppose, for now, instead of having non-identical servers and preference lists, when a call arrives we randomly sample servers until the first free server is found, and then we dispatch that server unless it does not violate the cutoff priority scheme. Let B_k denote the event that the k th selected server is busy (not available), $F_j \equiv B_k^c$ denote the event that the k th server selected is free (available), S_i denote the state of the queueing system where exactly i servers are busy (hence $P_i = P(S_i)$), and $P\{B_1 B_2 \cdots B_k F_{k+1}\}$ denote the probability that the first free server is the $k + 1$ st sampled server.

Under the cutoff scheme, if a low priority call arrives, the probability of dispatching the $k + 1$ st selected server is

$$P\{B_1 B_2 \cdots B_k F_{k+1}\} = P\{B_1 B_2 \cdots B_k F_{k+1} | S_i, i < s_C\},$$

since we do not select a server to dispatch to a low priority call if the number of busy servers is equal to or greater than the cutoff. Therefore we wish to derive an expression for $P\{B_1 B_2 \cdots B_k F_{k+1} | S_i, i < s_C\}$ that motivates an approximation procedure for the model in which servers are not identical. Recall that in the Hypercube model under a fixed-preference dispatching policy, the dispatcher always assigns the most preferred available server. Therefore the desired probability is the probability that the first k preferred servers are busy and the $k + 1$ st server is free. By conditioning on the number of busy servers, we obtain

$$P\{B_1 B_2 \cdots B_k F_{k+1} | S_i, i < s_C\} = \sum_{i=0}^{s_C-1} P\{B_1 B_2 \cdots B_k F_{k+1} | S_i\} P_i. \quad (32)$$

where

$$\begin{aligned} P\{B_1 B_2 \cdots B_k F_{k+1} | S_i\} \\ = P\{F_{k+1} | B_1 B_2 \cdots B_k S_i\} P\{B_k | B_1 B_2 \cdots B_{k-1} S_i\} \cdots P\{B_1 | S_i\}. \end{aligned} \quad (33)$$

Then, as in [26], we have

$$\begin{aligned} P\{B_i | B_1 B_2 \cdots B_{j-1} S_i\} &= \frac{i - (j-1)}{s - (j-1)}, \quad j = 1, 2, \dots, i+1, \\ P\{F_{k+1} | B_1 B_2 \cdots B_k S_i\} &= \frac{s-i}{s-k}, \quad k = 0, 1, \dots, i. \end{aligned} \quad (34)$$

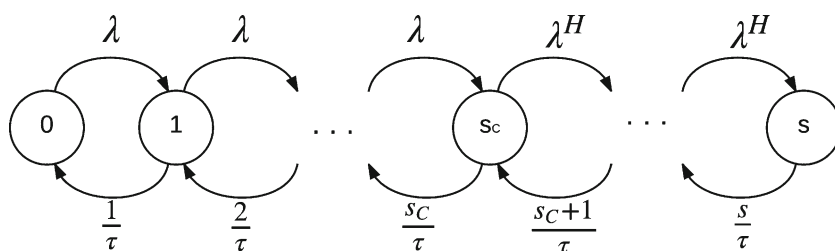
By substituting Eqs. 33 and 34 into Eq. 32 we get the expression for desired dispatch probability as

$$\begin{aligned} P\{B_1 B_2 \cdots B_k F_{k+1} | S_i, i < s_C\} &= \sum_{j=k}^{s_C-1} \frac{i - j + 1}{s - j + 1} \cdots \frac{i - (k-1)}{s - (k-1)} \frac{s-i}{s-k} P_i \\ &= \sum_{i=j}^{s_C-1} \frac{(s-k-1)!(s-i)!}{(i-k)!s!} P_i \end{aligned} \quad (35)$$

with P_i in Eqs. 5 and 6 (for the Loss model) or in Eqs. 7, 8 and 9 (for the Queued model). We can also write another expression for Eq. 35 using the correction factors:

$$P\{B_1 B_2 \cdots B_k F_{k+1} | S_i, i < s_C\} = q_k^L r^k (1 - r),$$

Fig. 9 Transition Diagram for a Zero Capacity Queue with a Cutoff s_C



where

$$q_k^L = \frac{\sum_{i=k}^{s_C-1} \frac{(s-k-1)!(s-i)i!}{(i-k)!s!} P_i}{(1 - P_s \frac{\lambda^H}{\lambda} - \sum_{i=s_C}^s P_i \frac{\lambda^L}{\lambda})^k \rho^k \left(1 - (1 - P_s \frac{\lambda^H}{\lambda} - \sum_{i=s_C}^s P_i \frac{\lambda^L}{\lambda}) \rho \right)}. \quad (36)$$

Lastly, the derivation of q_j^H is analogous to the above derivation with s_C replaced by s .

Appendix B: Computational results for large-scale dataset

Here we provide additional computational results that correspond to Tables 2–4 for the Scaled-up dataset. Tables 6

and 7 display dispatch probabilities for the Scaled-up dataset with 16 servers for selected number of s_R . The tables are truncated for $k \geq 5$, since the values when $k \geq 5$ are very small. Table 8 presents the server busy probabilities for the Scaled-up dataset. The left hand columns for all tables report the Hypercube model results, while the right hand columns report the simulation results. As in the original dataset (Tables 2–4), the discrepancy between the analytical Hypercube model results and simulation results are small.

Table 6 High Priority Dispatch Probabilities f_k^H and Loss Probabilities for High Priority Calls $P(\text{lost}|H)$ for Different Values of s_R in the Scaled-up Hanover County Dataset

	Hypercube					Simulation				
	$s_R = 0$	$s_R = 4$	$s_R = 8$	$s_R = 12$	$s_R = 15$	$s_R = 0$	$s_R = 4$	$s_R = 8$	$s_R = 12$	$s_R = 15$
$k = 1$	0.3685	0.4032	0.4945	0.5825	0.6096	0.3601	0.3957	0.4879	0.5802	0.6105
$k = 2$	0.2117	0.2300	0.2456	0.2295	0.2145	0.2005	0.2145	0.2206	0.2061	0.1961
$k = 3$	0.1288	0.1383	0.1317	0.1047	0.0936	0.1251	0.1314	0.1230	0.1002	0.0910
$k = 4$	0.0827	0.0878	0.0667	0.0461	0.0449	0.0834	0.0863	0.0690	0.0508	0.0478
$k = 5$	0.0533	0.0534	0.0331	0.0213	0.0208	0.0555	0.0543	0.0400	0.0278	0.0239
$P(\text{lost} H)$	0.0226	0.0013	0.0001	0.0000	0.0000	0.0244	0.0015	0.0001	0.0000	0.0000

Table 7 Low Priority Dispatch Probabilities f_k^L and Loss Probabilities for Low Priority Calls $P(\text{lost}|L)$ for Different Values of s_R in the Scaled-up Hanover County Dataset

	Hypercube					Simulation				
	$s_R = 0$	$s_R = 4$	$s_R = 8$	$s_R = 12$	$s_R = 15$	$s_R = 0$	$s_R = 4$	$s_R = 8$	$s_R = 12$	$s_R = 15$
$k = 1$	0.3939	0.3965	0.3197	0.1076	0.0043	0.3885	0.3916	0.3083	0.1027	0.0040
$k = 2$	0.2163	0.2057	0.1206	0.0175	0.0000	0.2127	0.2026	0.1189	0.0174	0.0000
$k = 3$	0.1307	0.1082	0.0442	0.0022	0.0000	0.1301	0.1076	0.0439	0.0022	0.0000
$k = 4$	0.0766	0.0573	0.0134	0.0001	0.0000	0.0770	0.0573	0.0138	0.0001	0.0000
$k = 5$	0.0489	0.0303	0.0038	0.0000	0.0000	0.0497	0.0309	0.0041	0.0000	0.0000
$P(\text{lost} L)$	0.0226	0.1737	0.4973	0.8725	0.9957	0.0242	0.1809	0.5098	0.8776	0.9960

Table 8 System-wide Server Busy Probability r for Different Values of s_R in the Scaled-up Hanover County Dataset

	Hypercube					Simulation				
	$s_R = 0$	$s_R = 4$	$s_R = 8$	$s_R = 12$	$s_R = 15$	$s_R = 0$	$s_R = 4$	$s_R = 8$	$s_R = 12$	$s_R = 15$
r	0.6119	0.5682	0.4603	0.3364	0.2950	0.6188	0.5731	0.4637	0.3394	0.3003

References

1. Ansari S, McLay LA, Mayorga ME (2015) A maximum expected covering problem for district design. *Transp Sci* 51(1):376–390
2. Aringhieri R, Bruni M, Khodaparasti S, van Essen J (2017) Emergency medical services and beyond: addressing new challenges through a wide literature review. *Comput Oper Res* 78:349–368
3. Aringhieri R, Carello G, Morale D (2016) Supporting decision making to improve the performance of an Italian emergency medical service. *Ann Oper Res* 236(1):131–148
4. Batta R, Dolan JM, Krishnamurthy NN (1989) The maximal expected covering location problem: Revisited. *Transp Sci* 23(4):277–287
5. Benn BA (1966) Hierarchical car pool systems in railroad transportation. Ph.D. thesis, Case Institute of Technology, Cleveland
6. Boyaci B, Geroliminis N (2015) Approximation methods for large-scale spatial queueing systems. *Transportation Res B Methodol* 74:151–181
7. Brotonne L, Laporte G, Semet F (2003) Ambulance location and relocation models. *Eur J Oper Res* 147(3):451–463
8. Budge S, Ingolfsson A, Erkut E (2009) Technical note—approximating vehicle dispatch probabilities for emergency service systems with location-specific service times and multiple units per location. *Oper Res* 57(1):251–255
9. Burwell TH, Jarvis JP, McKnew MA (1993) Modeling co-located servers and dispatchties in the hypercube model. *Comput Oper Res* 20(2):113–119
10. Cho SH, Jang H, Lee T, Turner J (2014) Simultaneous location of trauma centers and helicopters for emergency medical service planning. *Oper Res* 62(4):751–771
11. Church R, ReVelle C (1974) The maximal covering location problem. *Pap Reg Sci* 32(1):101–118
12. Daskin MS (1983) A maximum expected covering location model: formulation, properties and heuristic solution. *Transp Sci* 17(1):48–70
13. Figueira G, Almada-Lobo B (2014) Hybrid simulation—optimization methods: a taxonomy and discussion. *Simul Model Pract Theory* 46:118–134
14. Gendreau M, Laporte G, Semet F (2001) A dynamic model and parallel tabu search heuristic for real-time ambulance relocation. *Parallel Comput* 27(12):1641–1653
15. Geroliminis N, Karlaftis MG, Skabardonis A (2009) A spatial queueing model for the emergency vehicle districting and location problem. *Transp Res B Methodol* 43(7):798–811
16. Goldberg JB (2004) Operations research models for the deployment of emergency services vehicles. *EMS Manag J* 1(1):20–39
17. Iannoni AP, Morabito R (2007) A multiple dispatch and partial backup hypercube queueing model to analyze emergency medical systems on highways. *Transp Res E Logist Trans Rev* 43(6):755–771
18. Ingolfsson A, Budge S, Erkut E (2008) Optimal ambulance location with random delays and travel times. *Health Care Manag Sci* 11:262–274
19. Jagtenberg CJ, Bhulai S, der Mei RDV (2016) Dynamic ambulance dispatching: is the closest-idle policy always optimal? *Health care management science (to appear)*
20. Jaiswal NK (1968) Priority queues. Academic Press, New York
21. Jarvis JP (1985) Approximating the equilibrium behavior of multi-server loss systems. *Manag Sci* 31(2):235–239
22. Kim SH, Whitt W (2014) Are call center and hospital arrivals well modeled by non-homogeneous Poisson processes? *Manuf Serv Oper Manag* 16(3):464–480
23. Kolesar P, Walker WE (1974) An algorithm for the dynamic relocation of fire companies. *Oper Res* 22(2):249–274
24. Larsen MP, Eisenberg MS, Cummins RO, Hallstrom AP (1993) Predicting survival from out-of-hospital cardiac arrest: a graphic model. *Ann Emerg Med* 22(11):1652–1658
25. Larson RC (1974) A hypercube queueing model for facility location and redistricting in urban emergency services. *Oper Res* 1(1):67–95
26. Larson RC (1975) Approximating the performance of urban emergency service systems. *Oper Res* 23(5):845–868
27. Maxwell MS, Restrepo M, Henderson SG, Topaloglu H (2010) Approximate dynamic programming for ambulance redeployment. *INFORMS J Comput* 22(2):266–281
28. McLay LA (2009) A maximum expected covering location model with two types of servers. *IIE Trans* 41(8):730–741
29. McLay LA, Mayorga ME (2013) A Dispatching Model for Server-to-Customer Systems That Balances Efficiency and Equity. *Manuf Serv Oper Manag* 15(2):205–220
30. McLay LA, Moore H (2012) Hanover county improves its response to emergency medical 911 patients. *Interfaces* 42(4):380–394
31. Mendonça F, Morabito R (2001) Analysing emergency medical service ambulance deployment on a Brazilian highway using the hypercube model. *J Oper Res Soc*, pp 261–270
32. Rajagopalan H, Saydam C, Xiao J (2008) A multiperiod set covering location model for dynamic redeployment of ambulances. *Comput Oper Res* 35(3):814–826
33. Restrepo M, Henderson S, Topaloglu H (2009) Erlang loss models for the static deployment of ambulances. *Health Care Manag Sci* 12:67–79
34. Reuter-Oppermann M, van den Berg P, Vile J (2017) Logistics for emergency medical service systems. *Health Syst*. doi:[10.1057/s41306-017-0023-x](https://doi.org/10.1057/s41306-017-0023-x)
35. Schaack C, Larson RC (1986) An n-server cutoff priority queue. *Oper Res* 34(2):257–266
36. de Souza RM, Morabito R, Chiyoshi FY, Iannoni AP (2015) Incorporating priorities for waiting customers in the hypercube queueing model with application to an emergency medical service system in Brazil. *Eur J Oper Res* 242(1):274–285
37. Taylor IDS, Templeton JGC (1980) Waiting time in a multi-server cutoff-priority queue, and its application to an urban ambulance service. *Oper Res* 28(5):1168–1188